

PERFORMANCE EVALUATION AND DIMENSIONING OF NETWORKS AND SYSTEMS

METI

Lab Report [EN]

Authors:

José Oliveira (103771)

Tiago Videira (116069)

Afonso Mendes (116024)

jose.novais.oliveira@tecnico.ulisboa.pt

tiago.g.videira@tecnico.ulisboa.pt

afonsofmenDES@tecnico.ulisboa.pt

2026/2027 – 1st Semester

Contents

1	Introduction	2
2	A. Random number generators	3
2.1	Exercise 1	3
2.2	Exercise 2	3
3	B. Generation of random variables and stochastic processes	4
3.1	Exercise 3	4
3.2	Exercise 4	6
3.3	Exercise 5	7
4	C. Programming discrete event simulators	9
4.1	Exercise 6	9
4.2	Exercise 7	11
5	D. Estimation of probabilities	13
5.1	Exercise 8	13
6	E. Revisions of statistics	13
6.1	Exercise 9	13
6.2	Exercise 10	13
6.3	Exercise 11	13
7	F. Output data analysis	13
7.1	Exercise 12	13
7.2	Exercise 13	13
7.3	Exercise 14	13
8	Conclusion	13

1 Introduction

Text for the introduction here.

2 A. Random number generators

2.1 Exercise 1

Objective: Generate 1000 random numbers uniformly distributed in $[0, 1]$ using `runif()` and plot their histogram with `hist()`, to check informally whether the generator behaves as expected.

Approach: The following R script generates the sample and plots the histogram.

```
numbers <- runif(10000, min = 0, max = 1)
hist(numbers, main = "Histogram", xlab = "Value", ylab = "Samples",
      col = "lightblue", border = "black")
```

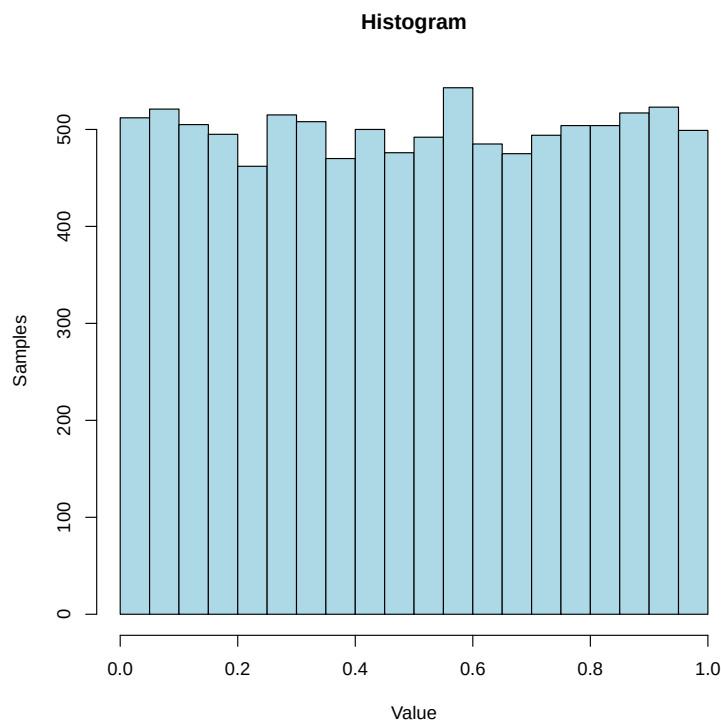


Figure 1: Histogram of 10000 samples generated with `runif()`.

Conclusion: As shown in Figure 1, the samples are approximately evenly distributed across all bins in $[0, 1]$, with no systematic bias toward any sub-interval. This is consistent with the expected behavior of a uniform distribution, confirming that `runif()` is generating numbers as intended.

2.2 Exercise 2

Objective: Verify, using `runif()`, that the sequence generated by a pseudo-random number generator is in fact deterministic, i.e., that it can be reproduced exactly.

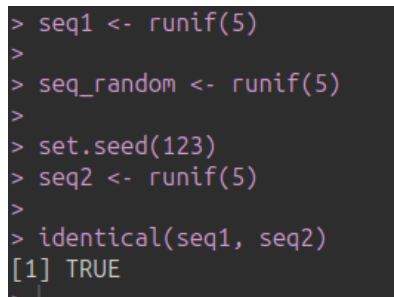
Approach: The `set.seed()` function fixes the internal state of R's random number generator. By setting the same seed before two separate calls to `runif()`, the two resulting sequences can be compared with `identical()` to check whether they match exactly. As a control, a third sequence is generated without resetting the seed, so it is expected to differ.

```
set.seed(123)
seq1 <- runif(5)

seq_random <- runif(5) # generated without resetting the seed

set.seed(123)
seq2 <- runif(5)

identical(seq1, seq2)
```



```
> seq1 <- runif(5)
>
> seq_random <- runif(5)
>
> set.seed(123)
> seq2 <- runif(5)
>
> identical(seq1, seq2)
[1] TRUE
>
```

Figure 2: R console output confirming that `seq1` and `seq2` are identical.

Conclusion: As shown in Figure 2, `identical(seq1, seq2)` returns `TRUE`, confirming that `seq1` and `seq2` are exactly the same sequence, even though they were generated by separate calls to `runif()`. This happens because `set.seed()` resets the internal state of the generator to the same fixed value before each call, so the pseudo-random number generator produces identical output whenever it starts from the same seed. This demonstrates that the sequence is not truly random but deterministic, fully determined by the seed. In contrast, `seq_random`, generated without resetting the seed, follows on from the generator's state after `seq1` and therefore differs from both.

3 B. Generation of random variables and stochastic processes

3.1 Exercise 3

Objective: Develop an R script to generate Bernoulli distributed numbers from a uniform random number generator (`runif()`), and plot a histogram of 1000 samples to informally check that the generator behaves correctly.

Approach: A Bernoulli random variable with success probability p can be generated from a uniform random variable $U \sim \text{Unif}(0, 1)$ by setting the outcome to 1 if $U < p$ and 0 otherwise, using the `ifelse()` function. Here $p = 0.7$ and $n = 1000$ samples were generated.

```
p <- 0.7 # success probability
n_samples <- 1000 # number of samples

u <- runif(n_samples)

bernoulli_numbers <- ifelse(u < p, 1, 0)

hist(bernoulli_numbers,
     breaks = c(-0.5, 0.5, 1.5),
     main = "Bernoulli Numbers Histogram (p = 0.7)",
     xlab = "Outcome",
     ylab = "Frequency",
     col = "lightblue",
     xaxt = "n")

axis(1, at = c(0, 1))
```

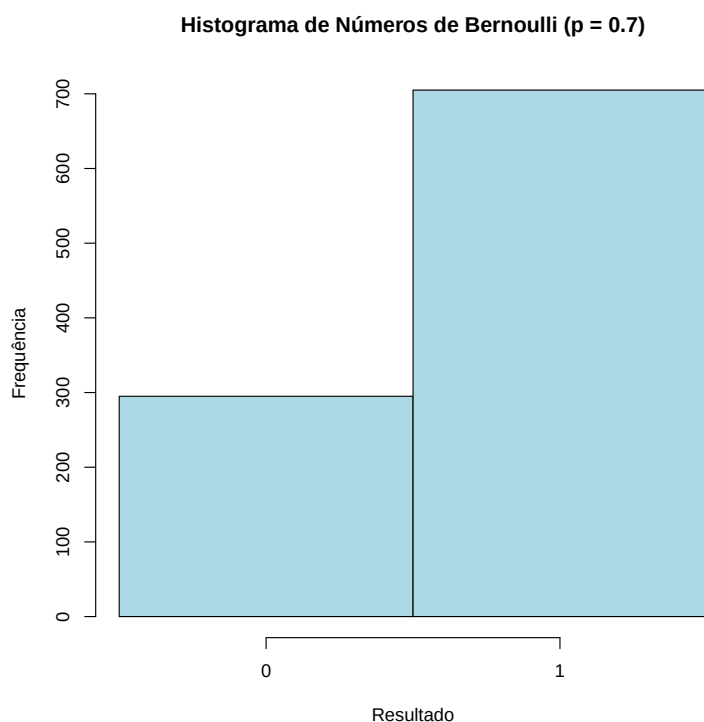


Figure 3: Histogram of 1000 Bernoulli samples with $p = 0.7$.

Conclusion: As shown in Figure 3, out of 1000 samples approximately 700 outcomes are 1

and 300 are 0, matching the expected proportions for $p = 0.7$. This confirms that thresholding a uniform random variable at p is a valid method for generating Bernoulli distributed samples, since $P(U < p) = p$ for $U \sim \text{Unif}(0, 1)$.

3.2 Exercise 4

Objective: Develop an R script to generate exponentially distributed numbers from a uniform random number generator (`runif()`), and plot a histogram of 10000 samples superposed with the theoretical pdf, to informally check that the generator behaves correctly.

Approach: Exponentially distributed samples can be generated from a uniform random variable $U \sim \text{Unif}(0, 1)$ via the inverse transform method: since the exponential CDF is $F(x) = 1 - e^{-\lambda x}$, setting $U = F(X)$ and solving for X gives $X = -\ln(1 - U)/\lambda$. The histogram is plotted with `prob=TRUE` so that its area is normalized to 1, making it directly comparable to the theoretical pdf obtained with `dexp()`, which is superposed using `lines()`. Here $\lambda = 1.5$ and $n = 10000$ samples were generated.

```
lambda <- 1.5 # rate of the exponential distribution
n_samples <- 10000 # number of samples

u <- runif(n_samples)
exp_numbers <- -log(1 - u) / lambda

hist(exp_numbers,
      breaks = 50,
      prob = TRUE,
      main = "Exponential Distribution (lambda = 1.5)",
      xlab = "Value",
      ylab = "Density",
      col = "lightgray",
      border = "white")

x_vals <- seq(0, max(exp_numbers), length.out = 500)
pdf_vals <- dexp(x_vals, rate = lambda)

lines(x_vals, pdf_vals, col = "red", lwd = 2)
```

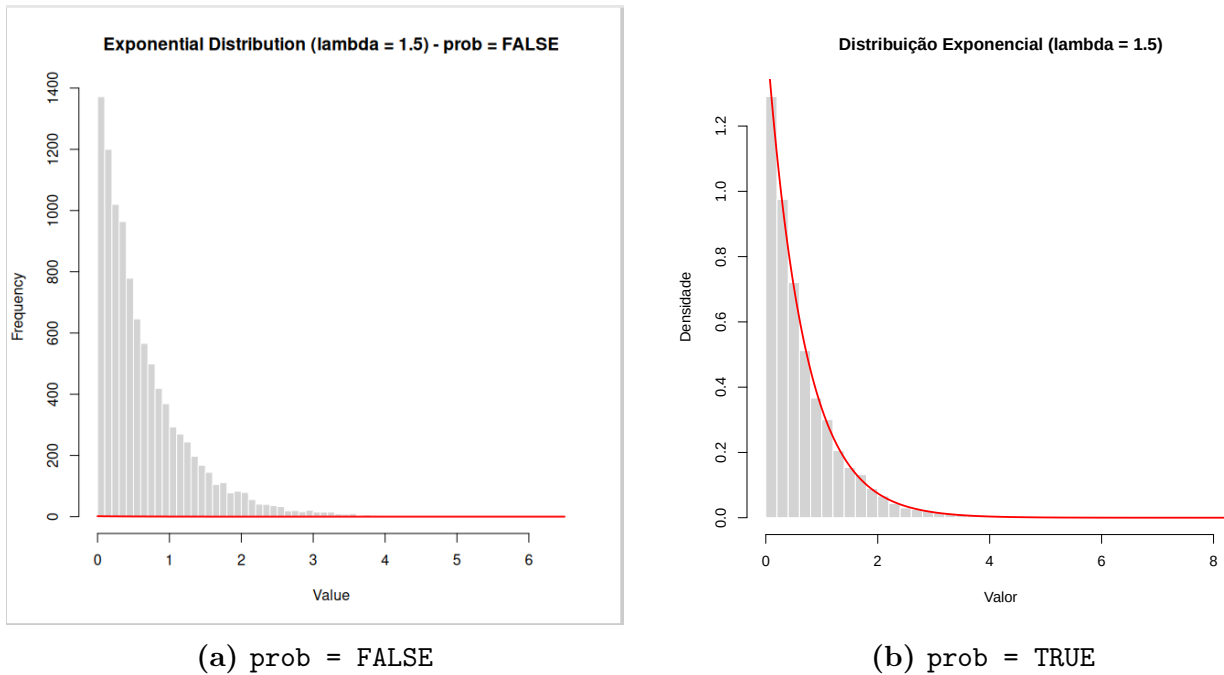


Figure 4: Histogram of 10000 exponentially distributed samples ($\lambda = 1.5$) with the theoretical pdf (red line) superposed, with and without density normalization.

Conclusion: Figure 4 shows why the `prob=TRUE` option is essential for this comparison. In Figure 4a (`prob=FALSE`), the histogram bars represent raw counts per bin, whose total sums to $n = 10000$, while the `dexp()` curve has area 1 by definition; the two are on completely different scales, so the pdf line appears flat against the axis and gives no useful comparison. In Figure 4b (`prob=TRUE`), the histogram is rescaled so that the total area of the bars is also 1, matching the normalization of the theoretical pdf, and the empirical histogram now closely follows the red curve. This confirms that the inverse transform method correctly generates exponentially distributed samples from uniform random numbers, and that density normalization is a necessary condition for a visual histogram-vs-pdf comparison to be meaningful.

3.3 Exercise 5

Objective: Develop an R script to generate the arrival times of a Poisson process, and perform a simple test to verify that the sample rate matches the population rate.

Approach: A Poisson process with rate λ has exponentially distributed inter-arrival times with parameter λ . Thus, inter-arrival times can be generated directly with `rexp()`, and the arrival times are obtained by accumulating them with `cumsum()`. To verify the generator, the sample rate is estimated from the generated data and compared with the population rate λ used to generate it.


```

lambda <- 2.0
n_arrivals <- 10000

inter_arrival_times <- rexp(n_arrivals, rate = lambda)

arrival_times <- cumsum(inter_arrival_times)

estimated_lambda <- 1/mean(inter_arrival_times)

cat("Population rate (lambda):", lambda, "\n")
cat("Estimated rate (sample):", estimated_lambda, "\n")

head(arrival_times, 10)

```

```

> n_arrivals <- 10000
>
> inter_arrival_times <- rexp(n_arrivals, rate = lambda)
>
> arrival_times <- cumsum(inter_arrival_times)
>
> estimated_lambda <- 1/mean(inter_arrival_times)
>
> cat("Taxa da população (lambda):", lambda, "\n")
Taxa da população (lambda): 2
> cat("Taxa estimada (amostral):", estimated_lambda, "\n")
Taxa estimada (amostral): 1.992062
>
> head(arrival_times, 10)
[1] 0.5065964 0.7858169 0.8949484 1.0201424 1.2737779 2.4506971 2.4521745 2.7405388 3.3030483
[10] 3.4816601
>

```

Figure 5: R console output showing the population rate, the estimated rate, and the first 10 arrival times.

Conclusion: As shown in Figure 5, with a population rate of $\lambda = 2$, the estimated rate from the sample is $\hat{\lambda} \approx 1.992$, very close to the true value. This result follows directly from the definition of λ : it represents the average number of arrivals per unit time, i.e., the rate of the process. Its reciprocal, $1/\lambda$, is the mean time between consecutive arrivals – the expected value of the exponential inter-arrival time distribution, $E[T] = 1/\lambda$. Since the sample mean of the generated inter-arrival times, $\bar{T}$, is an estimator of $E[T]$, taking its reciprocal, $1/\bar{T}$, naturally produces an estimator of λ itself. In other words, λ measures “events per unit time” while $E[T]$ measures “time per event”, so the two quantities are inverses of each other by construction, and computing $1/\bar{T}$ recovers an estimate of the rate that generated the data. The close agreement between $\hat{\lambda}$ and λ confirms that the inter-arrival times were correctly generated as exponentially distributed with rate λ .

4 C. Programming discrete event simulators

4.1 Exercise 6

Objective: Modify the M/M/1 simulator of Annex A to also estimate the average delay in system and the server utilization, and compare the results against the theoretical values (Annex B), for at least two values of λ/μ with $\lambda/\mu < 1$.

Approach: Starting from the simulator in Annex A, the following changes were made to also track the average delay in system and the server utilization, in addition to the average delay in queue already computed by the original code:

- Four new variables were initialized before the main loop: `AcumSystemTime=0` and `InServiceArrivalTime=0` to compute the delay in system (as described previously), and `BusyTime=0` together with `LastEventTime=0`, to track server utilization.
- At the very start of each loop iteration, right after `Time` is updated and *before* branching on the event type, the interval since the last event is added to `BusyTime` if the server was busy during that interval. This must be done unconditionally for both arrivals and departures, since the server can remain busy across either type of event:

```
NextEventType=which.min(EventList)
Time=EventList[NextEventType]
if (ServerStatus==1) { # NEW
  BusyTime=BusyTime+(Time-LastEventTime) # NEW
} # NEW
LastEventTime=Time # NEW
```

- In the arrival branch, when the server is free, the customer's arrival time is stored in `InServiceArrivalTime` (as before):

```
ServerStatus=1
InServiceArrivalTime=Time # NEW
EventList[2]=Time+rexp(1,ServiceRate)
```

- In the departure branch, the sojourn time of the departing customer is added to `AcumSystemTime`, and `InServiceArrivalTime` is updated if another customer enters service:

```

AcumSystemTime=AcumSystemTime+Time-InServiceArrivalTime # NEW
NumQueueCompleted=NumQueueCompleted+1
if (NumInQueue==0) {
    ServerStatus=0
    EventList[2]=Inf
} else {
    AcumDelay=AcumDelay+Time-QueueArrivalTime[1]
    InServiceArrivalTime=QueueArrivalTime[1] # NEW
    QueueArrivalTime=QueueArrivalTime[-1]
    NumInQueue=NumInQueue-1
    EventList[2]=Time+rexp(1,ServiceRate)
}

```

- After the loop, the average delay in system and the server utilization are computed alongside the average delay in queue:

```

AvgDelay=AcumDelay/NumQueueCompleted
AvgTimeInSystem=AcumSystemTime/NumQueueCompleted # NEW
Utilization=BusyTime/Time # NEW

```

The simulation was run with NumQueueCompleted = 1000 for three stable configurations ($\lambda/\mu < 1$) and, for comparison, one critically loaded configuration ($\lambda/\mu = 1$). The theoretical values were computed from Annex B: $W_Q = \frac{\lambda}{\mu(\mu - \lambda)}$, $W = \frac{1}{\mu - \lambda}$, and $\rho = \lambda/\mu$.

λ	μ	W_Q (sim.)	W_Q (theo.)	W (sim.)	W (theo.)	ρ (sim.)	ρ (theo.)
1	2	0.4572	0.5000	0.9473	1.0000	0.5421	0.5000
1	3	0.1678	0.1667	0.5142	0.5000	0.2975	0.3333
1	4	0.0727	0.0833	0.3186	0.3333	0.2732	0.2500
2	2	3.3556	∞	3.8212	∞	0.9939	1.0000

Table 1: Simulated vs. theoretical average delay in queue (W_Q), average delay in system (W), and server utilization (ρ), for different λ/μ ratios.

Conclusion: As shown in Table 1, for all stable configurations ($\rho < 1$) the simulated W_Q , W , and ρ closely match the theoretical values predicted by the M/M/1 formulas, with the small discrepancies attributable to the finite sample size (1000 completed customers) and the inherent variability of the simulation. Both the delays and the utilization increase as λ/μ approaches 1: for the M/M/1 system, $\rho = \lambda/\mu$ is precisely the theoretical fraction of time the server is busy, and this matches well against `BusyTime/Time` measured directly from the simulated trajectory. The last row ($\lambda = \mu = 2$, $\rho = 1$) illustrates the boundary of stability: the theoretical delays are infinite in the steady state, since the system is critically loaded (arrivals occur, on average, as fast as they can be served), yet the simulation returns large but finite delay values together with a simulated utilization close to 1 (0.9939). This finite delay is not a valid estimate of a steady-state quantity; it simply reflects that, over a finite run of 1000 customers, the queue built

up substantially – with the server almost always busy – without the delay yet diverging. This confirms that the condition $\rho < 1$ required by the exercise is essential for the M/M/1 system to reach a stable steady state, and shows that server utilization, unlike the mean queuing delay, remains a well-defined and finite quantity even at the stability boundary.

4.2 Exercise 7

Objective: Develop a simulator for estimating the average delay in queue of an M/M/2 system, and compare the results with the theoretical values (Annex C), for at least two values of $\lambda/(2\mu)$ with $\lambda/(2\mu) < 1$.

Approach: The M/M/1 simulator of Annex A was extended to two parallel servers. The key changes are: **ServerStatus** becomes a length-2 vector (one entry per server), and the event list has three entries instead of two – one for arrivals and one departure event per server. On arrival, if both servers are busy the customer joins the queue; otherwise the customer is assigned to the first free server found with `which(ServerStatus==0)[1]`. On a departure, the freed server either goes idle (if the queue is empty) or immediately picks up the next customer in queue, and the corresponding delay is accumulated exactly as in the M/M/1 case.

```

ArrivalRate=1
ServiceRate=2
Time=0
NumQueueCompleted=0
ServerStatus=c(0,0) # 0 = idle, 1 = busy, one entry per server
NumInQueue=0
AcumDelay=0
QueueArrivalTime=c()
EventList=c(rexp(1,ArrivalRate), Inf, Inf) # [arrival, dep@server1, dep@server2]

while (NumQueueCompleted<1000) {
  NextEventType=which.min(EventList)
  Time=EventList[NextEventType]

  if (NextEventType==1) { # ARRIVAL
    EventList[1]=Time+rexp(1,ArrivalRate)
    if (all(ServerStatus==1)) { # both servers busy -> join queue
      QueueArrivalTime=c(QueueArrivalTime,Time)
      NumInQueue=NumInQueue+1
    } else { # a server is free -> start service
      freeServer=which(ServerStatus==0)[1]
      ServerStatus[freeServer]=1
      NumQueueCompleted=NumQueueCompleted+1
      EventList[freeServer+1]=Time+rexp(1,ServiceRate)
    }
  } else { # DEPARTURE from server (NextEventType-1)
    server=NextEventType-1
    if (NumInQueue==0) {
      ServerStatus[server]=0
      EventList[NextEventType]=Inf
    } else {
      AcumDelay=AcumDelay+Time-QueueArrivalTime[1]
      QueueArrivalTime=QueueArrivalTime[-1]
      NumInQueue=NumInQueue-1
      NumQueueCompleted=NumQueueCompleted+1
      EventList[NextEventType]=Time+rexp(1,ServiceRate)
    }
  }
}

AvgDelay=AcumDelay/NumQueueCompleted

```

The simulation was run with $\text{NumQueueCompleted} = 1000$ for two configurations satisfying $\rho = \lambda/(2\mu) < 1$. The theoretical average delay in queue was computed from Annex C: $W_Q = P_Q \frac{\rho}{\lambda(1-\rho)}$, where P_Q is the probability that an arrival finds both servers busy (Erlang C formula, $m = 2$).

λ	μ	$\rho = \lambda/(2\mu)$	W_Q (sim.)	W_Q (theo.)
1	2	0.25	0.0556	0.0333
2	2	0.50	0.1380	0.1667

Table 2: Simulated vs. theoretical average delay in queue (W_Q) for the M/M/2 system, for different $\lambda/(2\mu)$ ratios.

Conclusion: As shown in Table 2, the simulated W_Q is reasonably close to the theoretical value in both configurations, with the discrepancy attributable to the finite sample size (1000 completed customers) and the inherent variability of a single simulation run – this is discussed further in Exercise 12, which shows that a single run is generally not enough to draw firm conclusions. As with the M/M/1 system, the average delay in queue increases with $\rho = \lambda/(2\mu)$, since higher load makes it more likely that an arriving customer finds both servers busy. Comparing with Exercise 6, for a similar load level the M/M/2 system exhibits substantially lower delays than the corresponding M/M/1 system (e.g., $\rho = 0.5$ gives $W_Q \approx 0.17$ here, versus $W_Q \approx 0.50$ for M/M/1 with $\lambda = 1, \mu = 2$), illustrating the benefit of server pooling: two servers each handling half the traffic intensity of an equivalent single fast server yield lower expected waiting times, since the probability of both being simultaneously busy is lower than the probability of a single server being busy.

5 D. Estimation of probabilities

5.1 Exercise 8

6 E. Revisions of statistics

6.1 Exercise 9

6.2 Exercise 10

6.3 Exercise 11

7 F. Output data analysis

7.1 Exercise 12

7.2 Exercise 13

7.3 Exercise 14

8 Conclusion

Text for the conclusion here.